Machine Learning Interview Knowledge

Core ML concepts, evaluation, debugging and deployment reasoning

Purpose
Supervised/unsupervised/RL, model families, preprocessing, metrics, validation, leakage, imbalance and deployment.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. Problem framing
2. 2. Train, validation and test
3. 3. Overfitting, underfitting and bias/variance
4. 4. Regression and classification
5. 5. Trees, random forests, SVM and clustering
6. 6. Metrics
7. 7. Cross-validation and tuning
8. 8. Leakage, imbalance and debugging
9. 9. Deployment and monitoring

1. Problem framing

Supervised learning uses labeled examples. Unsupervised learning looks for structure without a target label. Reinforcement learning learns through interaction and rewards.

Before choosing an algorithm, define the prediction unit, target, features available at prediction time, success metric, latency/cost constraints and failure consequences.

2. Train, validation and test

Training data fits model parameters. Validation data supports model selection and hyperparameter decisions. A final test set estimates generalization after choices are made. If the test set repeatedly influences tuning, it is no longer a clean final estimate.

Cross-validation can provide a more stable estimate on moderate datasets. Preprocessing must be fit on training data only, then applied to validation/test data.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3. Overfitting, underfitting and bias/variance

Underfitting means the model cannot capture enough useful structure; both training and validation performance can be poor. Overfitting means training performance is strong but generalization is weak.

Bias/variance is a conceptual lens: high bias often means excessive simplicity; high variance means high sensitivity to the training sample. Regularization, data quantity, features and model capacity can change the balance.

Symptom	Interpretation	Possible action
Train poor, validation poor	Underfitting/high bias	Features/capacity/less regularization
Train strong, validation weak	Overfitting/high variance	More data/regularization/simpler model
Scores vary greatly by split	Small/noisy data	Better validation/data collection

4. Regression and classification

Linear regression predicts a continuous value using a linear combination of features. Logistic regression models class probabilities and is a strong interpretable baseline for many classification tasks.

A probability threshold converts a score into a class. Changing the threshold changes precision/recall, so the default 0.5 threshold is not universally optimal.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]
pred = (proba >= 0.5).astype(int)
```

5. Trees, random forests, SVM and clustering

Decision trees represent nonlinear rules but can overfit. Random forests average many varied trees and commonly reduce variance compared with one deep tree.

SVMs optimize a margin and can use kernels for nonlinear boundaries. Feature scaling can matter. K-means assigns observations to nearby centroids and updates centroids iteratively; it requires a chosen k and does not guarantee business-meaningful clusters.

Model	Strength	Caution
Linear/logistic	Simple baseline	Limited nonlinear representation
Decision tree	Readable nonlinear rules	Can overfit
Random forest	Robust nonlinear baseline	Less transparent/larger
SVM	Strong margin/kernel method	Scaling/compute
K-means	Simple clustering	Needs k and suitable geometry

6. Metrics

Accuracy is the fraction of correct predictions. Precision is TP/(TP+FP). Recall is TP/(TP+FN). F1 is the harmonic mean of precision and recall. ROC-AUC measures ranking performance across thresholds.

Metric choice follows error costs. In rare-event detection, accuracy can be misleading; inspect precision/recall and the confusion matrix, and consider PR-oriented evaluation.

Metric	Question it answers
Accuracy	How many predictions were correct overall?
Precision	When I predict positive, how often am I right?
Recall	Of actual positives, how many did I find?
F1	How well do precision and recall balance?
ROC-AUC	How well does the model rank positives above negatives?

7. Cross-validation and tuning

K-fold cross-validation trains on k-1 folds and validates on the remaining fold repeatedly. Stratification is useful for many classification tasks so class proportions remain more stable across folds.

Grid search evaluates a defined parameter grid; randomized search samples configurations. Keep the final test set out of repeated tuning.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
    LogisticRegression(max_iter=1000), X_train, y_train,
    cv=cv, scoring="f1"
)
```

8. Leakage, imbalance and debugging

Leakage occurs when information unavailable at prediction time enters training, including target-derived information leaking through preprocessing or feature construction. Leakage can create excellent offline scores and poor production behavior.

For class imbalance, use metrics aligned with the rare class, stratified splits and, when justified, class weights or sampling. Always validate against the real objective.

9. Deployment and monitoring

A model artifact includes learned parameters plus preprocessing and configuration. Version them together so the production input transformation matches training.

Monitor input quality, latency, error rates and prediction distributions. When labels arrive, monitor real performance and drift. A model can degrade without application code changing.

```
# Illustrative deployment contract
{
  "model_version": "2026-10-01",
  "features": ["age", "income", "tenure"],
  "threshold": 0.42,
  "preprocessing_version": "v3"
```

}

Scenario: validation F1 is excellent, production F1 is poor

Situation: Investigate whether the split resembles production, preprocessing was fit only on training data, labels changed, future information leaked into features, or production class prevalence differs.

Reasoning: Reproduce production-like validation before increasing model complexity.

Scenario: 98% accuracy but most fraud is missed

Situation: A rare positive class can be overwhelmed by negatives. Inspect the confusion matrix, recall and precision; define the cost of false negatives and choose a threshold accordingly.

Reasoning: Reporting accuracy alone.

Interview question

Why split train/validation/test?

Strong answer: To separate learning, model selection and final evaluation. Otherwise repeated choices can overfit the evaluation data.

Follow-up: When would cross-validation help?

Interview question

What is leakage?

Strong answer: Information available only after the prediction point, or information derived from the target/evaluation set, enters training. It makes offline performance unrealistically strong.

Follow-up: Give a preprocessing example.

Interview question

Precision versus recall?

Strong answer: Precision emphasizes the correctness of positive predictions; recall emphasizes finding actual positives. The right trade-off depends on the cost of false positives and false negatives.

Follow-up: How does threshold affect them?

Interview question

Why can random forests be robust?

Strong answer: A forest aggregates many varied trees, reducing dependence on one training sample and capturing nonlinear interactions.

Follow-up: What is the cost?

Interview question

What is underfitting?

Strong answer: The model is too constrained or the features are inadequate to capture useful patterns, leading to poor training performance and usually poor validation performance.

Follow-up: How would you diagnose it?

Interview question

How do you debug poor model performance?

Strong answer: First verify labels and splits, then baseline performance, feature quality, leakage, missing values, preprocessing, class distribution, metric choice and threshold. Only after data/pipeline correctness should you escalate model complexity.

Follow-up: Why start with a baseline?

Revision checklist

- Frame the problem and metric before modeling.
- Separate train/validation/test responsibilities.

- Fit preprocessing on training data only.
- Explain overfitting, underfitting and bias/variance.
- Know regression/classification differences.
- Explain trees, forests, SVM and clustering.
- Interpret confusion matrices.
- Use cross-validation appropriately.
- Recognize leakage and imbalance.
- Describe deployment and monitoring.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://scikit-learn.org/stable/user_guide.html

https://scikit-learn.org/stable/model_selection.html

End of knowledge base document